

Roteiro de Prática — Publish-Subscribe e RPC em Go

Comunicação em grupo e chamada de procedimentos remotos FPPD — Unidade 5

Preparação

Faça download do pacote `fppd-pratica-pubsub-rpc.zip` no Moodle e descompacte. A estrutura é:

<code>fppd-pratica-pubsub-rpc/</code>	
├── <code>exerciciol-chat/</code>	Chat distribuído com NATS (pub-sub)
│ ├── <code>chat.go</code>	Chat funcional – executar e explorar
│ └── <code>teste-conexao.go</code>	Script de teste de conectividade
├── <code>exercicio2-rpc/</code>	Chamada de procedimentos remotos (RPC)
│ ├── <code>servidor.go</code>	Servidor RPC – adicionar <code>SomarIntervalo</code>
│ └── <code>cliente.go</code>	Cliente RPC – adicionar chamada a <code>SomarIntervalo</code>
└── <code>go.mod</code>	Módulo Go (dependências já configuradas)

Instalar dependências

Na pasta do projeto, execute:

```
cd fppd-pratica-pubsub-rpc
go mod download
```

Testar conectividade com o servidor NATS

Antes de começar os exercícios, verifique se você consegue acessar o servidor NATS:

```
go run exerciciol-chat/teste-conexao.go
```

Saída esperada:

```
Testando conexão com demo.nats.io...
Conectado! Servidor: nats://demo.nats.io:4222
Recebido: hello from FPPD!
Pub/Sub funcionando corretamente!
```

Se não conseguir conectar: verifique se sua rede permite conexões TCP na porta 4222. Em redes corporativas com firewall restritivo, pode ser necessário usar a rede Wi-Fi do celular (hotspot) como alternativa.

Conceitos-chave

Antes de começar, relembre os conceitos de **publish-subscribe**:

- **Publisher** — publica mensagens em um **subject** (tópico) sem conhecer os destinatários
- **Subscriber** — se inscreve em subjects de interesse e recebe mensagens automaticamente
- **Broker** — intermedia a comunicação, mantendo o registro de inscrições (no nosso caso, o servidor NATS)
- **Desacoplamento espacial** — publisher e subscriber não precisam se conhecer
- **Desacoplamento temporal** — no NATS básico, as mensagens são entregues apenas em tempo real; se um subscriber não estiver conectado, ele não recebe as mensagens enviadas nesse período

O NATS usa **subjects** hierárquicos separados por . (ponto). Exemplo: `fppd.turma32.chat`. Cada subject funciona como um grupo — quem se inscreve naquele subject recebe as mensagens publicadas nele.

Exercício 1 — Chat distribuído com NATS

Objetivo: explorar na prática o modelo publish-subscribe usando um chat em grupo com NATS, onde todos os participantes de uma sala recebem as mensagens uns dos outros em tempo real.

Parte A — Executar e explorar o chat

O arquivo `exercicio1-chat/chat.go` contém um chat funcional e pronto para uso. Leia o código e entenda a estrutura antes de executar. O programa:

1. Conecta ao servidor NATS público (`demo.nats.io`)
2. Inscreve-se (**subscribe**) em um **subject** (tópico) da sala
3. Publica (**publish**) mensagens digitadas no mesmo subject
4. Recebe mensagens de todos os outros participantes inscritos

A **sala** é definida pelo primeiro argumento de linha de comando. O professor informará o nome da sala da turma.

Passo 1 — Execute o chat em dois terminais:

```
# Terminal 1
go run exercicio1-chat/chat.go turma32 Alice

# Terminal 2
go run exercicio1-chat/chat.go turma32 Bob
```

Passo 2 — Experimente:

- Digite mensagens em um terminal e observe a recepção no outro
- Quando seus colegas se conectarem à mesma sala, você verá as mensagens deles
- Pressione Ctrl+C para sair e observe a notificação de saída nos outros terminais

Passo 3 — Explore o código-fonte. Identifique no `chat.go`:

- a) Qual linha realiza a **conexão** ao broker NATS?
- b) Qual linha faz o **subscribe** (inscrição no tópico)?
- c) Qual linha faz o **publish** (publicação de mensagem)?
- d) Qual é o **subject** usado? Como ele é montado?
- e) Como o programa diferencia mensagens próprias de mensagens de outros participantes?

Parte B — Extensão: mensagens privadas

Atualmente, todas as mensagens são públicas — todos na sala veem tudo. Sua tarefa é adicionar um comando `/privado` que envia uma mensagem diretamente para um participante específico, sem que os demais vejam.

Requisitos:

- a) Se o usuário digitar `/privado <destinatario> <mensagem>`, a mensagem deve ser enviada em um subject separado que só o destinatário recebe. Sugestão de subject: `fppd.<sala>.dm.<destinatario>`.
- b) Cada participante deve se inscrever também no subject de mensagens privadas dirigidas a ele (`fppd.<sala>.dm.<seunome>`), além do subject da sala.
- c) Mensagens privadas recebidas devem ser exibidas de forma diferenciada, por exemplo: `[HH:MM:SS] (privado) Alice: mensagem`.
- d) Mensagens normais (sem o prefixo `/privado`) devem continuar funcionando como antes.

Dicas:

- Use `strings.HasPrefix(texto, "/privado ")` para detectar o comando
- Use `strings.SplitN(texto, " ", 3)` para separar o comando, destinatário e mensagem
- Adicione um segundo `nc.Subscribe()` para o subject de DMs

Salve sua solução como `chat.go` (editando o arquivo existente).

Questões para reflexão (Exercício 1)

Responda as seguintes perguntas com base na sua experiência com o chat:

- Q1.** No modelo publish-subscribe, o publisher precisa saber quem são os subscribers? E vice-versa? Como isso se relaciona com o conceito de **desacoplamento espacial**?
- Q2.** Se você mudar o nome da sala (primeiro argumento), ainda recebe as mensagens dos colegas? Por quê? Relacione com a noção de **grupos** em comunicação em grupo.
- Q3.** Na extensão de mensagens privadas, o subject `fppd.<sala>.dm.<nome>` implementa comunicação ponto-a-ponto sobre um sistema pub-sub. Qualquer processo

pode se inscrever nesse subject sem autorização. Isso caracteriza um **grupo aberto ou fechado**? Quais são as implicações de segurança?

Exercício 2 — Chamada de procedimentos remotos (RPC)

Objetivo: experimentar a mecânica de RPC na prática — um servidor expõe funções, um cliente as invoca pela rede como se fossem chamadas normais — e estender o serviço com uma nova operação.

Conceitos-chave

Relembre os conceitos de **RPC (Remote Procedure Call)**:

- **Servidor** — registra um serviço com métodos que podem ser invocados remotamente
- **Cliente** — conecta ao servidor e chama métodos via `client.Call("Servico.Metodo", args, &reply)`
- **Serialização** — os argumentos e a resposta são empacotados/desempacotados automaticamente pelo framework (encoding/gob no net/rpc)
- **Tipos compartilhados** — cliente e servidor precisam concordar nos tipos dos argumentos e respostas

Parte A — Executar e explorar

O exercício contém dois arquivos:

- **exercicio2-rpc/servidor.go** — servidor RPC que expõe um `NumericaService` com duas operações: `ContarPrimos(inicio, fim)` e `EstimarPi(iteracoes)`. Também contém um TODO para `SomarIntervalo`.
- **exercicio2-rpc/cliente.go** — cliente que conecta ao servidor e chama as duas operações remotamente. Também contém um TODO para chamar `SomarIntervalo`.

Passo 1 — Inicie o servidor em um terminal:

```
go run exercicio2-rpc/servidor.go
```

Passo 2 — Execute o cliente em outro terminal:

```
go run exercicio2-rpc/cliente.go
```

Saída esperada no cliente:

Conectando ao servidor RPC em localhost:9000...
Conectado!

```
ContarPrimos(1, 100000) = 9592  
EstimarPi(1000000)      = 3.139740
```

`SomarIntervalo`: não implementado (TODO)

Observe que o servidor imprime cada chamada recebida no seu terminal.

Passo 3 — Explore o código-fonte. Identifique:

- a) No `servidor.go`: como o serviço é registrado? Qual a assinatura que um método precisa ter para ser exposto via RPC?
- b) No `cliente.go`: como a conexão é estabelecida? Como uma chamada remota é feita com `client.Call`?
- c) Os tipos `Intervalo`, `ArgIteracoes` e `ResultadoPi` aparecem nos dois arquivos. Por que precisam ser iguais?

Parte B — Extensão: adicionar `SomarIntervalo`

Sua tarefa é adicionar a operação `SomarIntervalo` ao serviço, modificando **ambos** os arquivos:

No servidor (`servidor.go`): implemente o método seguindo o TODO. A assinatura deve ser:

```
func (s *NumericaService) SomarIntervalo(args *Intervalo, reply *int64) error
```

A função deve somar todos os inteiros de `args.Inicio` até `args.Fim` (inclusive). Siga o padrão de `ContarPrimos`.

No cliente (`cliente.go`): adicione a chamada remota seguindo o TODO. Use `client.Call("NumericaService.SomarIntervalo", ...)` com os mesmos padrões das chamadas existentes.

Como verificar

Com o servidor atualizado rodando, execute o cliente. Saída esperada:

```
Conectando ao servidor RPC em localhost:9000...
Conectado!
```

```
ContarPrimos(1, 100000) = 9592
EstimarPi(1000000)      = 3.139740
SomarIntervalo(1, 1000) = 500500
```

Requisitos

- a) `SomarIntervalo` deve estar implementado no servidor e chamado no cliente.
- b) O servidor deve imprimir a chamada recebida (como as demais operações fazem).
- c) O resultado de `SomarIntervalo(1, 1000)` deve ser 500500.

Salve suas soluções editando os arquivos `servidor.go` e `cliente.go`.

Questões para reflexão (Exercício 2)

Q4. Se o servidor for desligado enquanto o cliente tenta fazer uma chamada, o que acontece? Isso seria possível em uma chamada de função local? O que isso diz sobre a **transparência semântica** em RPC?

Q5. A serialização do `net/rpc` usa `encoding/gob`, que é específico de Go. Isso permite que um cliente em Python chame este servidor? Qual é a vantagem de formatos como Protocol Buffers ou JSON?

Q6. As operações `ContarPrimos` e `SomarIntervalo` recebem um intervalo `[inicio, fim]`. Imagine que você quisesse acelerar `ContarPrimos(1, 1_000_000)` distribuindo o trabalho entre 4 servidores RPC, cada um processando uma faixa. Descreva em alto nível como faria isso e qual seria a vantagem.

Entrega

Envie pelo Moodle **3 arquivos**:

1. **chat.go** — Exercício 1 (chat com extensão de mensagens privadas)
 2. **servidor.go** e **cliente.go** — Exercício 2 (RPC com `SomarIntervalo`)
 3. **respostas.txt** — Respostas das questões Q1-Q6
-

Dicas gerais

- Use `go run -race arquivo.go` para verificar condições de corrida
- O servidor `demo.nats.io` é público e compartilhado — use um nome de sala único para a turma
- Se a conexão NATS falhar, verifique: (1) acesso à internet, (2) porta 4222 não bloqueada por firewall
- A biblioteca NATS para Go reconecta automaticamente em caso de queda temporária
- O pacote `net/rpc` de Go é simples mas limitado a clientes Go (serialização `gob`)
- Documentação do NATS: <https://docs.nats.io>
- Documentação do `net/rpc`: <https://pkg.go.dev/net/rpc>
- Documentação do cliente NATS Go: <https://pkg.go.dev/github.com/nats-io/nats.go>